TABLE

Restaurant Ordering System

Final project report | Java desktop application

Prepared: 5 November 2026

Team names: _____
Course / section: _____
Instructor: _____

Implementation and verification package. Add team details and actual individual contributions before submission. The suggested three-member split is a plan, not a claim of authorship.

Technology	Scope
Java 17+ / Swing	Two tabs, in-memory orders, six course patterns plus State
Storage	No database; orders disappear when the app closes
External services	None; no real payments or network calls

1. Introduction and requirements

The application models a small restaurant workflow: a customer configures meals, reviews a cart, selects a discount, and submits an order. A kitchen view advances the order through preparation. Both views display the same order status through event notifications.

The objective is to demonstrate six coordinated course patterns in a cohesive program while keeping the product small enough to explain and defend. The counted patterns are Factory Method, Decorator, Builder, Strategy, Observer, and Facade. State remains additional and is not counted toward the required six. The project uses a local Swing window rather than a website.

Requirement	Implementation / evidence
At least six distinct working patterns	Factory Method, Decorator, Builder, Strategy, Observer, Facade; see sections 3-5 and source appendix. State is additional.
At least two patterns per student	Proposed split in section 6; every student must independently implement their assigned patterns and record actual work.
One integrated application	Meal customization -> checkout -> order queue -> kitchen states -> customer notifications.
Report with code, screenshots and UML	This report includes introductory/body/conclusion/further-work sections, screenshots, diagrams, and full production Java source.
PDF and zipped source submission	output/pdf/restaurant-project-report.pdf and dist/restaurant-source.zip.
Oral defense	Walkthrough and questions in docs/defense.md.
SOLID in final report	Sections 9.1 and 9.2 provide code evidence and limitations.

Sources: Final Project Defense Requirements.pdf and Syllabus_Software Design Patterns.pdf, supplied by the team. The syllabus specifies a team of 2-3, PDF and ZIP submission by the end of Week 9, and defense in Week 10. Check Moodle for the exact deadline.

The syllabus permits AI for reference examples and clarification, not as a replacement for independent work. This generated implementation/report is reference material; it does not establish independent student implementation or authorship.

2. Features, boundaries, and rules

The Place Order tab offers pizza (2,500 KZT), burger (1,800 KZT), and lemonade (600 KZT). Food can be wrapped with cheese (+300 KZT), sauce (+150 KZT), or both. Lemonade disables food extras. Each Add action contributes one serving; repeated items represent multiple servings.

The cart supports item removal and one discount policy: none, student (10%), or promotion (15%). Discounts do not stack. Values use integer tenge; discount amounts round down to a whole tenge. Student eligibility is not verified in this classroom demonstration.

Submitting validates that the order is nonempty, preserves item descriptions/prices and calculated totals, allocates an ID, and notifies both views. It records an order; it does not charge money. The customer can select previous receipts. The kitchen can select any order and advance it once per click.

Lifecycle: Pending -> Preparing -> Ready -> Completed. The terminal action is disabled in the UI and rejected in the model. No backward transition, arbitrary status setter, or skipped step is exposed.

Orders remain in memory during one session. Restarting the application resets orders and IDs. There are no accounts, database, delivery tracking, reservations, external APIs, or payment services. All model operations are performed synchronously on the Swing event dispatch thread; the service is not designed as a concurrent backend.

3. Design patterns: creation and customization

Factory Method. MealCreator defines a common prepare() operation that invokes createMeal(). PizzaCreator, BurgerCreator, and DrinkCreator override this method and supply distinct Meal products. This is polymorphic construction, rather than a switch-based simple factory.

Decorator. Extra implements Meal and holds a wrapped Meal. Cheese and Sauce delegate to the wrapped object and extend description() and price(). A decorator can wrap another decorator, allowing combinations without new product subclasses for each combination.

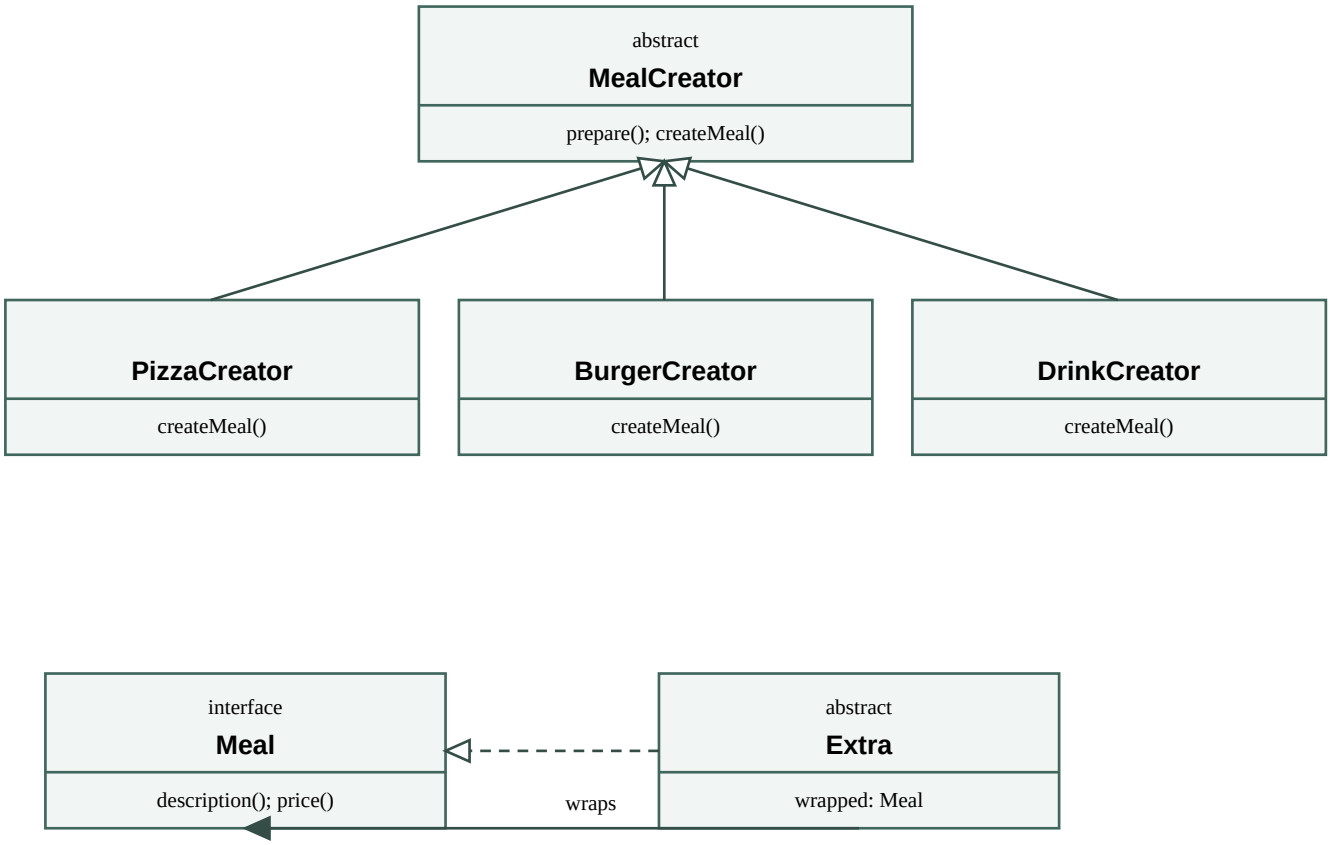


Figure 1. Factory Method and Decorator class relationships. Open triangles indicate generalization or interface realization (dashed). Cheese and Sauce extend Extra; Pizza, Burger, and Drink implement Meal. The complete editable diagram includes these products.

3.1 Builder: step-by-step order assembly

Builder is the sixth counted course pattern. OrderBuilder captures meals one at a time through addMeal(), selects an optional policy through withDiscount(), and produces a complete Order through build(id). Both configuration methods return the same builder for a fluent API. The default policy is no discount.

The facade creates and configures the builder. The service supplies the next session ID and calls build() before storing or notifying. The builder rejects empty orders and invalid IDs, validates discount bounds, and calculates totals. A failed build does not consume an ID or publish an event.

Each added meal is captured as an immutable item snapshot. Order defensively copies the list on construction, so further builder changes cannot alter an already built receipt. This separates construction from the Order representation and replaces a positional construction call in the service.

```
builder = new OrderBuilder()
    .addMeal(configuredPizza)
    .addMeal(lemonade)
    .withDiscount(new DiscountStrategy.Student());
order = builder.build(id);
```

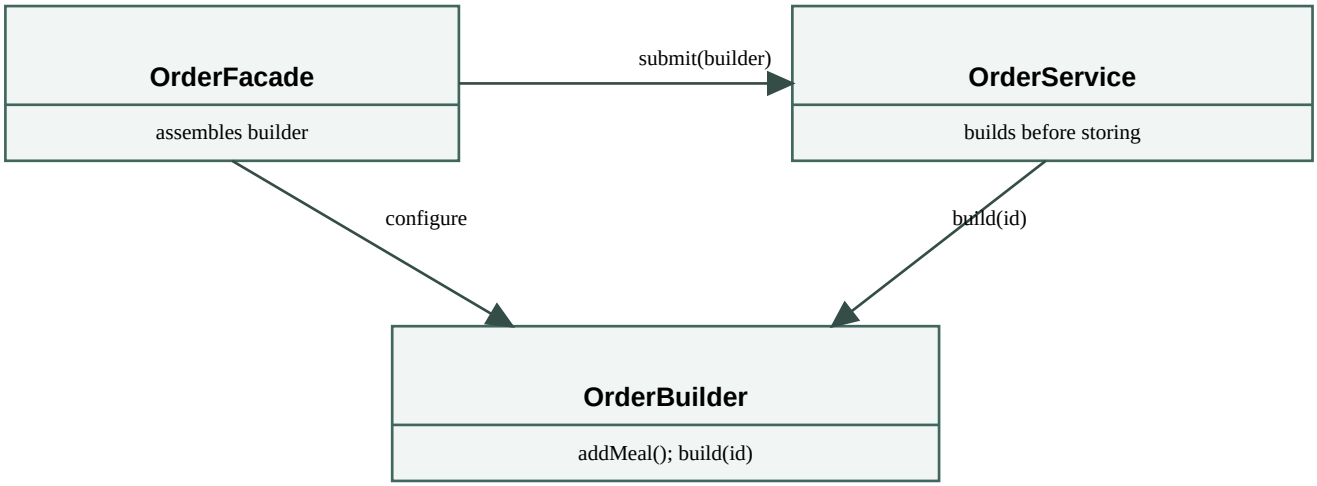


Figure 1b. Builder integration. OrderBuilder uses Meal and DiscountStrategy abstractions, builds Order, and leaves persistence/notification responsibilities to OrderService. A separate Director class is unnecessary: the facade coordinates construction.

4. Design patterns: checkout and lifecycle

Strategy. DiscountStrategy defines discount(subtotal). None, Student, and Promotion implement separate policies. OrderFacade accepts the chosen policy and uses it without knowing its concrete type. The same cart can be quoted with different policies.

State (additional; not counted toward six). Order owns an OrderState and delegates advancement to next(). Each concrete state returns its sole valid successor. Completed throws an exception and reports canAdvance() as false. The lifecycle rules are therefore in state classes rather than scattered across UI conditions.

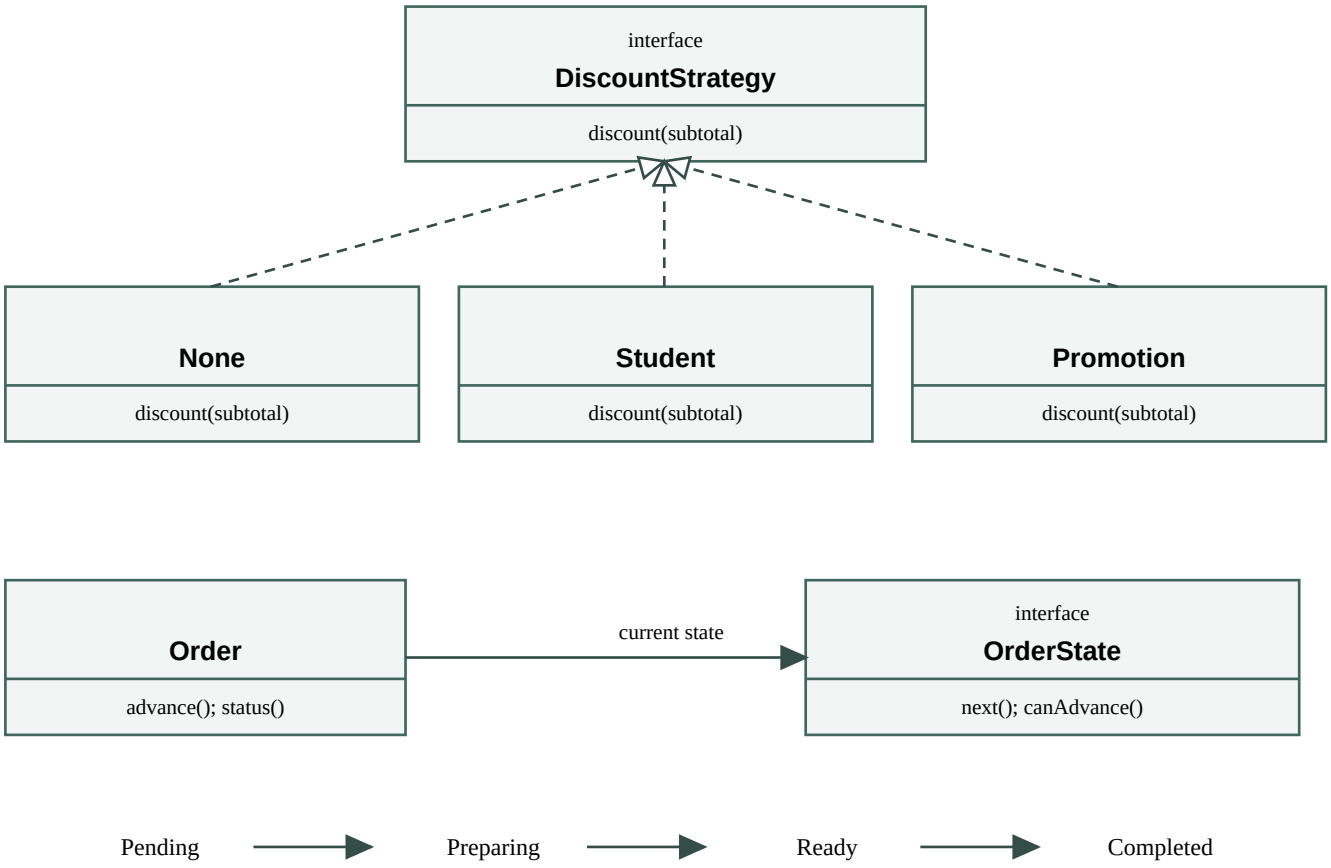


Figure 2. Strategy hierarchy and State context relationship. The four named states implement OrderState. The lifecycle permits only forward advancement to the next state.

5. Design patterns: orchestration and notifications

Facade. OrderFacade provides quote() and placeOrder() as simple client operations. It assembles OrderBuilder, then asks OrderService to build, store, and publish the order. Builder owns snapshots, validation, and pricing through Strategy. The facade hides this coordination from the UI without adding out-of-scope payment behavior.

Observer. OrderService stores OrderObserver subscribers and calls orderChanged(order) after successful submissions and transitions. RestaurantPanel registers separate customer and kitchen method-reference handlers. Each refreshes its own list and selected receipt. Both observe the same publisher even when only one tab is visible.

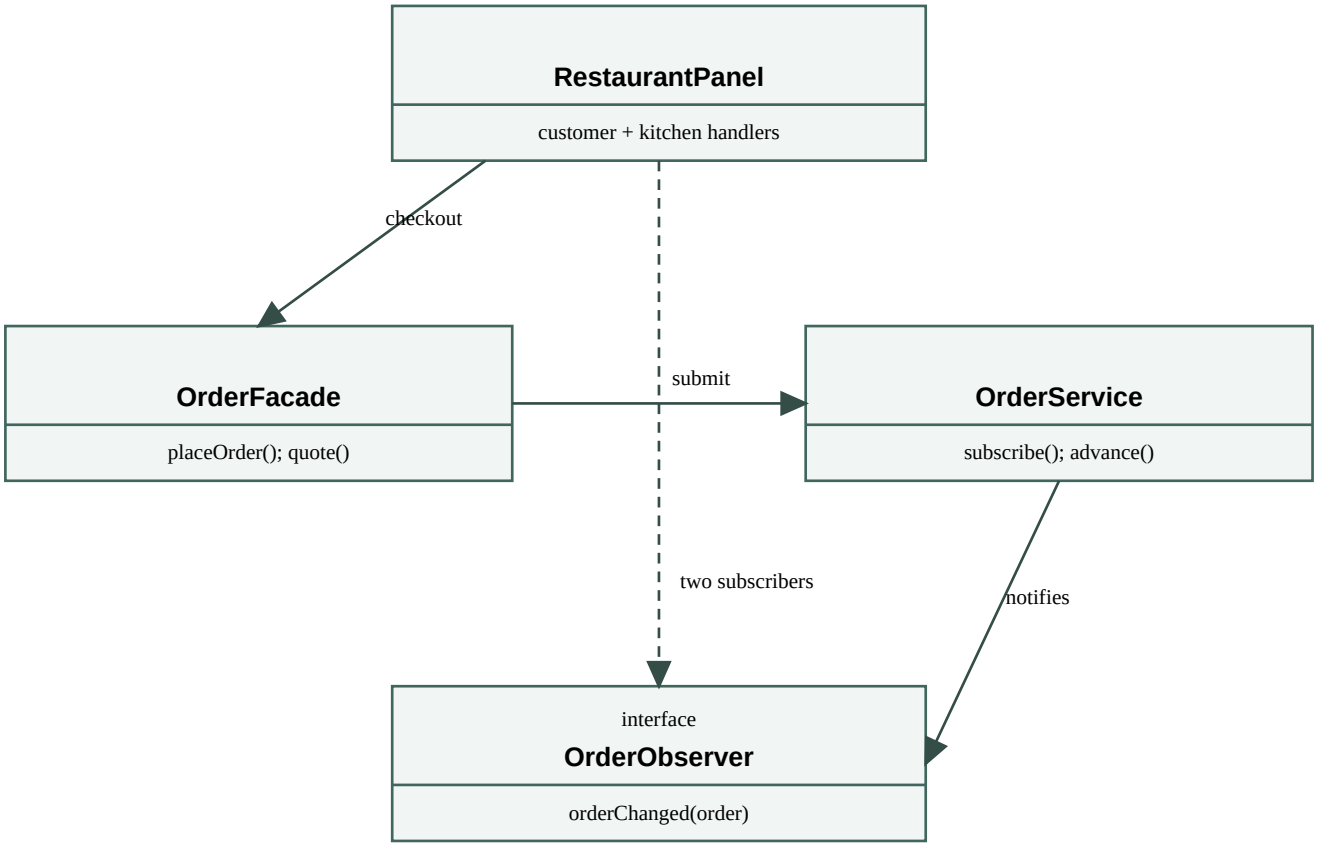


Figure 3. Facade and Observer collaboration. UI handlers conform to OrderObserver using Java method references; there are two independent subscriptions. OrderService also stores the session orders.

6. Integrated workflow and contribution plan

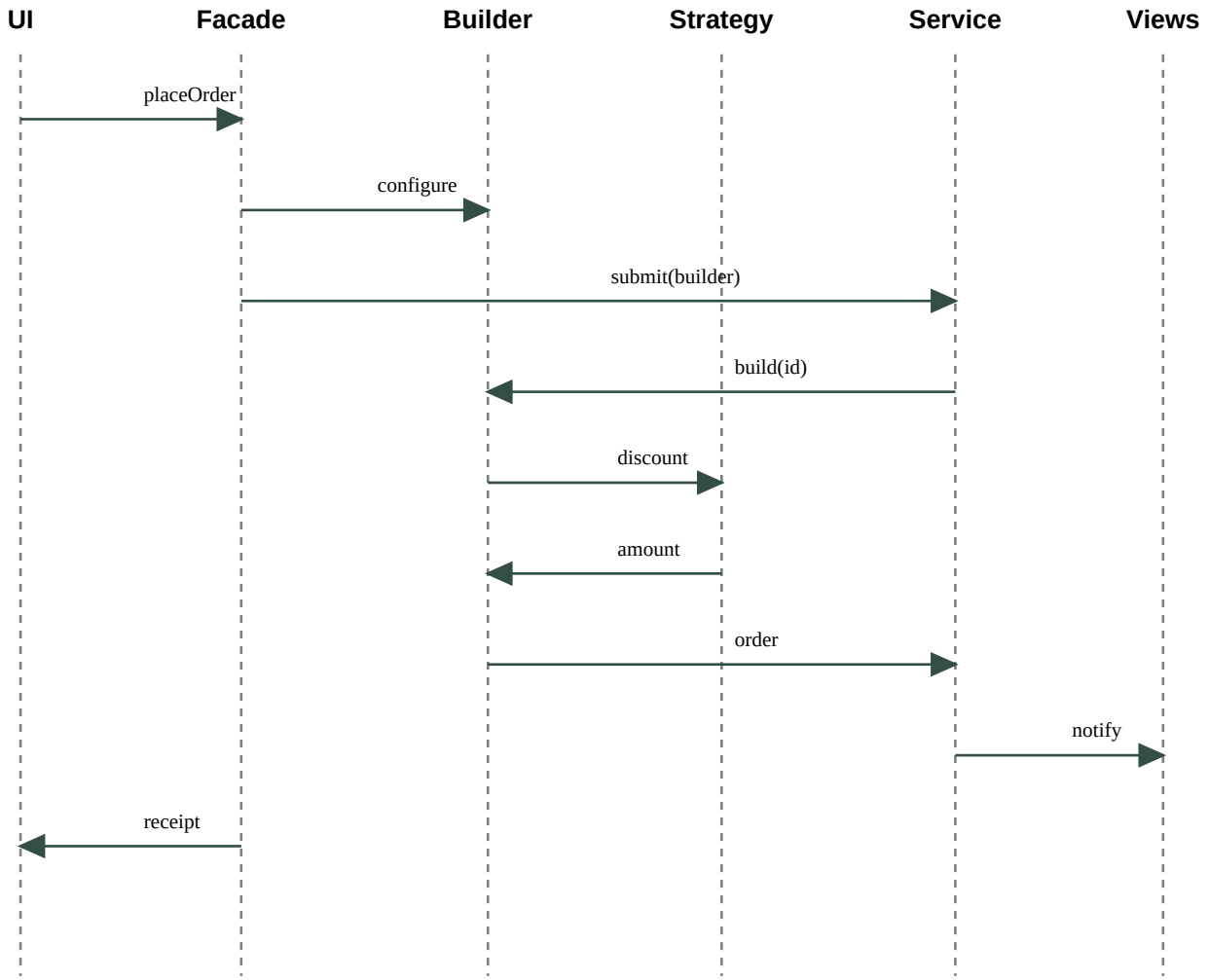


Figure 4. Submission sequence (simplified). Configure means add meals and select a discount policy. Views represents both subscribers. Service returns the built order through Facade to UI; the full editable sequence includes return messages and the additional State interaction.

On a kitchen action, UI calls OrderService.advance(id), the service finds Order, Order delegates to its current State, and the service then notifies both views. Invalid IDs and terminal transitions produce no event.

Proposed member	Counted patterns	Feature area
1 - name pending	Factory Method + Decorator	Menu and meal extras
2 - name pending	Builder + Strategy	Assembly and pricing
3 - name pending	Facade + Observer	Submission and updates

This allocation assumes three students. Replace placeholders with real names and document actual implementation work. Every student must individually implement at least two patterns. A different team size needs a reviewed allocation.

7. Application view: Place Order

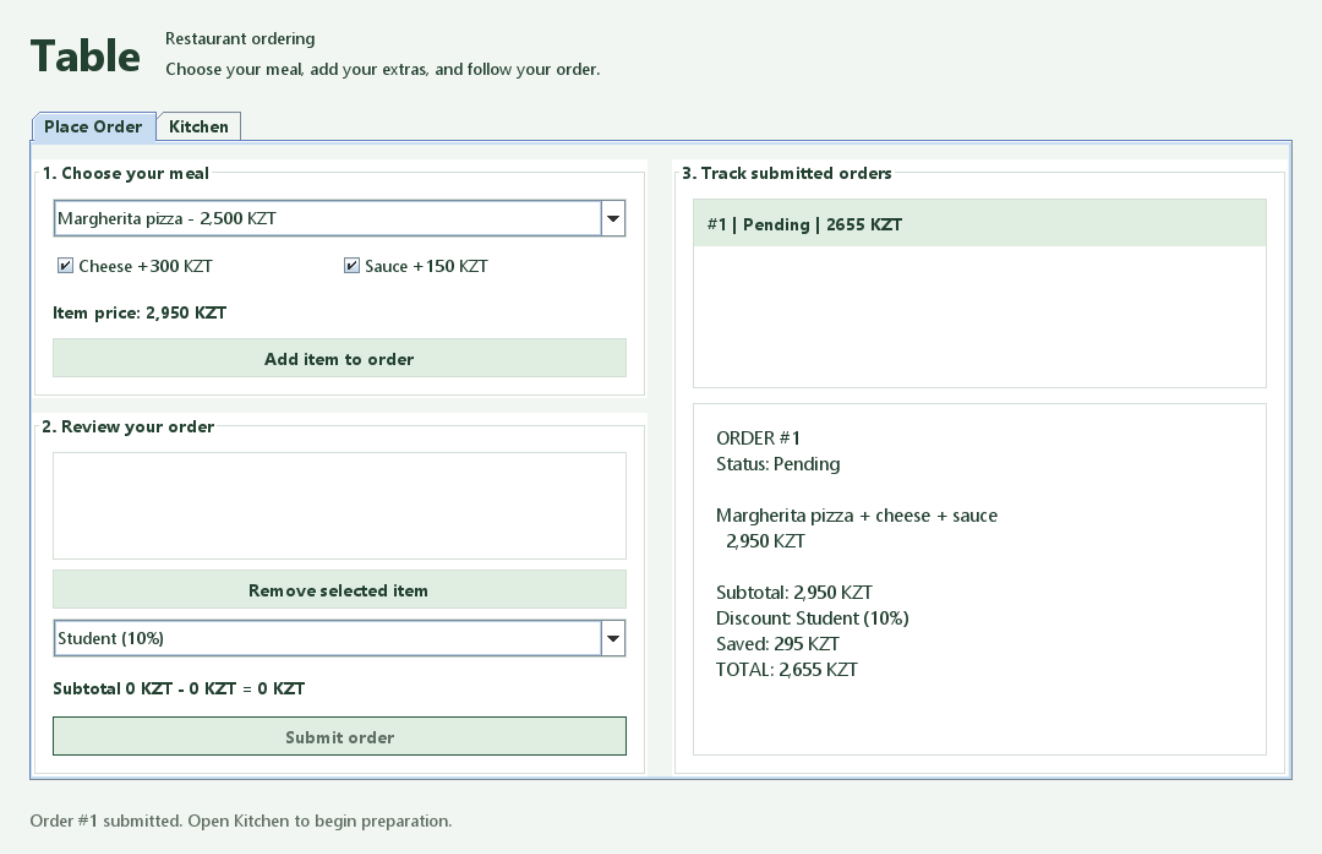


Figure 5. Real Swing panel rendered offscreen by the integration check. A pizza with both extras has been submitted under the student discount: 2,950 - 295 = 2,655 KZT. The cart is now empty and the Pending receipt is preserved.

Use the two tabs in one local window. No browser, server, database, network connection, or special credentials are required.

8. Application view: Kitchen

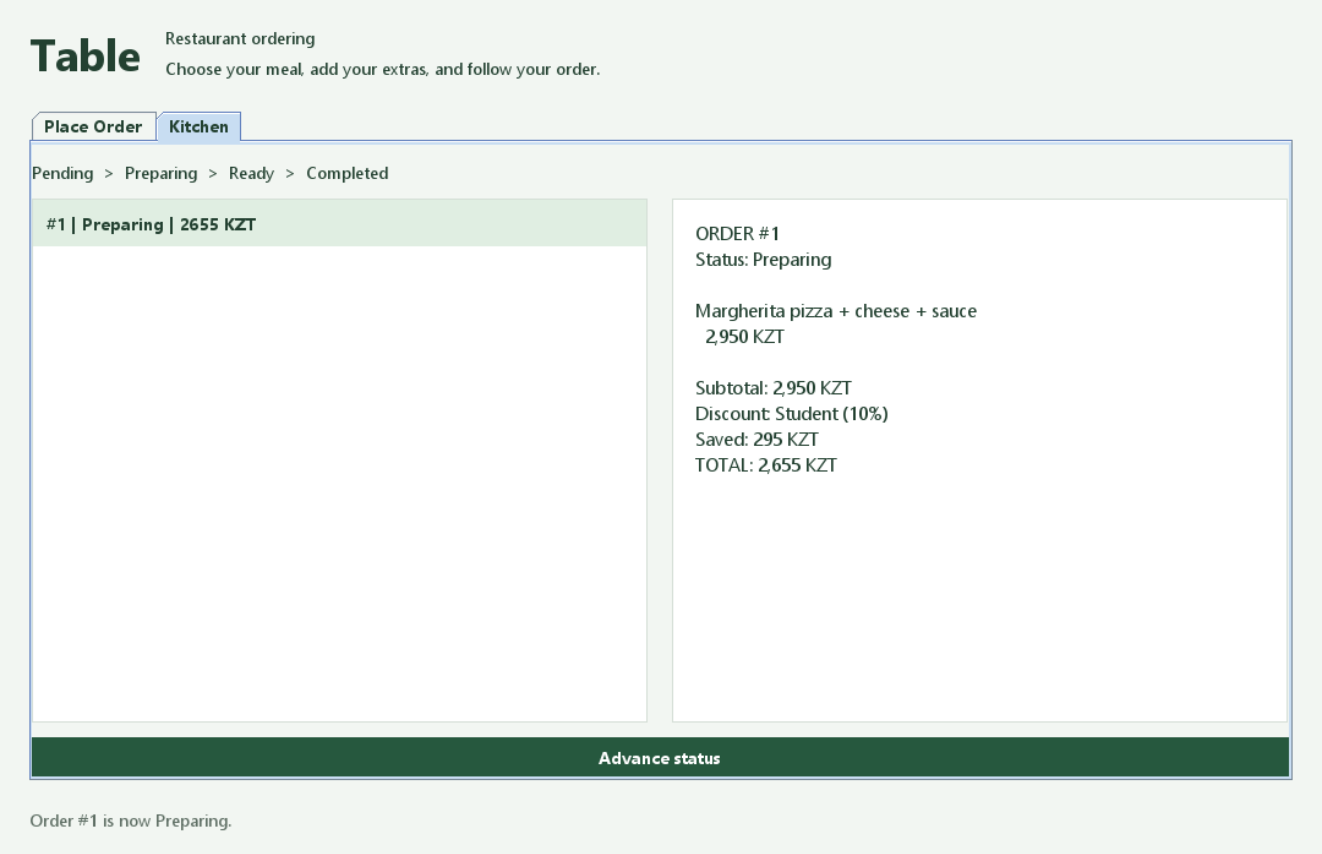


Figure 6. The same order after advancing to Preparing. The automated check confirms the customer receipt updates as well. These are offscreen application-panel renders, not native desktop-window captures.

Use the two tabs in one local window. No browser, server, database, network connection, or special credentials are required.

9. Verification and running the project

Prerequisite: JDK 17 or newer, with java, javac, and jar on PATH. Build and checks were executed with JDK 25.0.1 while producing Java 17 bytecode. No external Java libraries or build-system downloads are required.

Run from PowerShell:

```
powershell -ExecutionPolicy Bypass -File .\scripts\run.ps1
```

Verify:

```
powershell -ExecutionPolicy Bypass -File .\scripts\test.ps1
```

The verification suite passed 35 behavioral checks. It exercises menu products, stacked extras, all discount policies and rounding, empty-order rejection, immutable snapshots and collections, independent order states, terminal/unknown-order rejection, notification delivery and unsubscribe, invalid discounts, builder defaults/staged construction/reuse/validation, and actual Swing control interactions. It renders both application panels for visual inspection.

The Swing checks submit a configured pizza, verify totals and cart clearing, inspect both view receipts, advance through all states, verify the disabled terminal button, reset extras for drinks, remove an item, and submit a second independent order. Test failures cause a nonzero exit code.

A separate DesktopSmokeTest passed on Java 17: Main created a visible native Swing window at 1080 x 760, laid out its content at the minimum window size, and closed successfully. Both the headless behavioral suite and native startup check ran without external Java libraries.

Verification limitation: automated control interactions and native startup do not establish physical mouse/keyboard usability or behavior at every display scaling setting. Before defense, launch the application on the presentation machine and complete the manual smoke checklist in docs/defense.md. The full manual checklist has not been performed.

Editable UML sources: docs/uml/classes.puml, sequence.puml, and states.puml. The source archive contains Java production code, tests, scripts, documentation, screenshots, and this report. The executable JAR is supplied separately.

9.1 SOLID principles: evidence and limits

Single Responsibility. MealCreator creates meals; Extra customizes them; OrderBuilder assembles and validates orders; UiTheme styles controls. Business rules are outside Swing handlers. This is practical separation: RestaurantPanel still combines layout and interaction, and OrderService combines storage with publication.

Open/Closed. New DiscountStrategy or Extra implementations can extend pricing or customization without changing the builder pricing algorithm. A new concrete creator can supply another meal. The UI must still register a new choice; the project does not claim that adding features requires no edits anywhere.

Liskov Substitution. Plain meals and stacked decorators implement the same Meal contract and can be passed to addMeal(). All supplied discount strategies accept nonnegative subtotals and return an amount between zero and subtotal. Behavioral checks exercise both decorated meals and interchangeable policies; custom implementations must preserve those contracts.

Interface Segregation. Meal exposes only description and price; DiscountStrategy exposes a name and a calculation; OrderObserver exposes one callback. Pricing classes are not forced to implement UI or notification methods.

Dependency Inversion. Builder depends on Meal and DiscountStrategy abstractions; the publisher depends on OrderObserver; Order uses OrderState. The service is constructor-injected into the facade and panel. DIP is applied partially: the facade intentionally depends on concrete OrderService and constructs OrderBuilder. No unnecessary repository interface or injection framework is introduced.

These are code-backed design choices rather than a claim of perfect compliance. If the UI grows, separate customer/kitchen view classes; if storage requirements grow, reconsider a repository abstraction. Neither expansion is needed for the agreed scope.

9.2 Clean-code practices

Meaningful names and focused operations. placeOrder(), addMeal(), withDiscount(), canAdvance(), and orderChanged() state their purpose. The facade coordinates, the builder constructs, and policies calculate discounts. The largest class, RestaurantPanel, remains a possible future refactoring target.

DRY. Quote preview and order construction use the same OrderBuilder.quote() calculation. UI handlers do not repeat discount formulas. UiTheme centralizes visual choices and money() centralizes UI monetary formatting.

KISS and YAGNI. The implementation uses Swing, in-memory orders, integer tenge, and JDK-only checks. It avoids features outside the agreed scope: database, web server, accounts, payments, and external services.

Encapsulation and defensive copying. Order fields are private; submitted data fields are final; List.copyOf() protects item snapshots and returned order lists. Builder reuse cannot alter an existing receipt. Lifecycle state is intentionally mutable through controlled transitions.

Boundary validation and clear failures. Null inputs, invalid items, empty builds, invalid IDs, excessive discounts, and terminal advancement are rejected explicitly. Math.addExact() prevents silent subtotal overflow. Failed submission does not create or publish an order.

Composition and separation of concerns. Extras wrap meals rather than requiring a class for every combination. Strategies supply pricing behavior. The UI, theme, construction, pricing, and notification roles remain distinguishable and the business logic can be checked without a visible window.

Behavioral verification and useful documentation. Checks cover outcomes and boundaries, including pricing, snapshots, builder reuse, observers, lifecycle, and UI controls. Comments explain pattern intent; Markdown and UML explain usage and structure. This is not a claim of comprehensive JavaDoc or complete Clean Code compliance.

Known limits: the service is single-threaded and in memory; synchronous observer exceptions can interrupt subsequent delivery; source formatting is compact in places. More examples and limitations are documented in docs/solid-and-clean-code.md.

10. Conclusion and further work

The implemented system covers the agreed restaurant-ordering scope with a local desktop UI and no persistence layer. Six course patterns work as one workflow: factories create meals, decorators customize them, the facade assembles a builder, strategies calculate discounts, and observers refresh both views. State controls kitchen progression as an additional pattern and is not counted toward the six.

The main learning outcome is separation of responsibilities. Menu creation does not know pricing policies; the UI does not own transition rules; submitted receipts do not depend on a mutable cart; and notification subscribers do not control order storage.

Possible further work, outside the current project scope, includes an explicit quantity editor, cancellation with defined rules, additional meal types, and receipt export. A database or real payment integration is intentionally unnecessary for this defense.

Before submission: complete independent student implementation, fill team details, document actual contributions, complete the manual desktop check, and submit the PDF and source ZIP to Moodle by the stated deadline. This AI-generated reference is not evidence of independent work; follow the syllabus AI and authorship policies.

Defense example: pizza + cheese + sauce -> student discount -> submit -> Pending -> Preparing -> Ready -> Completed. Show matching customer/kitchen status, then demonstrate a second independent order. Each student should explain their two patterns and point to the relevant classes.

Appendix A. Complete production Java code

The following listings reproduce the production Java sources shipped with the project. Tests and PowerShell build scripts are included in the source archive. Long source lines are wrapped for printing only; use the .java files to compile.

DiscountStrategy.java

```
 1  package restaurant;
 2
 3  /** Strategy: return the discount amount, rounded down to whole tenge. */
 4  public interface DiscountStrategy {
 5      String name();
 6      int discount(int subtotal);
 7
 8      final class None implements DiscountStrategy {
 9          public String name() { return "No discount"; }
10          public int discount(int subtotal) { return 0; }
11          public String toString() { return name(); }
12      }
13      final class Student implements DiscountStrategy {
14          public String name() { return "Student (10%)"; }
15          public int discount(int subtotal) { return (int) ((long) subtotal * 10 / 100); }
16          public String toString() { return name(); }
17      }
18      final class Promotion implements DiscountStrategy {
19          public String name() { return "Promotion (15%)"; }
20          public int discount(int subtotal) { return (int) ((long) subtotal * 15 / 100); }
21          public String toString() { return name(); }
22      }
23  }
```

Extra.java

```
1  package restaurant;
2
3  import java.util.Objects;
4
5  /** Decorator: extras can wrap either a base meal or another extra. */
6  public abstract class Extra implements Meal {
7      protected final Meal wrapped;
8      protected Extra(Meal wrapped) { this.wrapped = Objects.requireNonNull(wrapped); }
9
10     public static final class Cheese extends Extra {
11         public Cheese(Meal meal) { super(meal); }
12         public String description() { return wrapped.description() + " + cheese"; }
13         public int price() { return wrapped.price() + 300; }
14     }
15     public static final class Sauce extends Extra {
16         public Sauce(Meal meal) { super(meal); }
17         public String description() { return wrapped.description() + " + sauce"; }
18         public int price() { return wrapped.price() + 150; }
19     }
20 }
```

Main.java

```
 1  package restaurant;
 2
 3  import javax.swing.*;
 4  import java.awt.*;
 5
 6  public final class Main {
 7      private Main() { }
 8      public static void main(String[] args) {
 9          SwingUtilities.invokeLater(() -> {
10              UIManager.put("Button.font", new Font(Font.SANS_SERIF, Font.PLAIN, 14));
11              UIManager.put("Label.font", new Font(Font.SANS_SERIF, Font.PLAIN, 14));
12              UIManager.put("List.font", new Font(Font.SANS_SERIF, Font.PLAIN, 14));
13              JFrame frame = new JFrame("TABLE - Restaurant Ordering System");
14              frame.setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
15              frame.setContentPane(new RestaurantPanel(new OrderService()));
16              frame.setSize(1080, 760);
17              frame.setMinimumSize(new Dimension(1000, 720));
18              frame.setLocationRelativeTo(null);
19              frame.setVisible(true);
20          });
21      }
22  }
```

Meal.java

```
1  package restaurant;
2
3  /** A configured item. Prices use integer tenge, avoiding floating point money. */
4  public interface Meal {
5      String description();
6      int price();
7  }
```

MealCreator.java

```
1  package restaurant;
2
3  /** Factory Method: subclasses decide which product the shared operation creates. */
4  public abstract class MealCreator {
5      protected abstract Meal createMeal();
6
7      public final Meal prepare() {
8          Meal meal = createMeal();
9          if (meal.price() < 0) throw new IllegalStateException("Invalid menu price");
10         return meal;
11     }
12
13     public static final class PizzaCreator extends MealCreator {
14         protected Meal createMeal() { return new Pizza(); }
15         public String toString() { return "Margherita pizza - 2,500 KZT"; }
16     }
17     public static final class BurgerCreator extends MealCreator {
18         protected Meal createMeal() { return new Burger(); }
19         public String toString() { return "Classic burger - 1,800 KZT"; }
20     }
21     public static final class DrinkCreator extends MealCreator {
22         protected Meal createMeal() { return new Drink(); }
23         public String toString() { return "Lemonade - 600 KZT"; }
24     }
25     private static final class Pizza implements Meal {
26         public String description() { return "Margherita pizza"; }
27         public int price() { return 2500; }
28     }
29     private static final class Burger implements Meal {
30         public String description() { return "Classic burger"; }
31         public int price() { return 1800; }
32     }
33     private static final class Drink implements Meal {
34         public String description() { return "Lemonade"; }
35         public int price() { return 600; }
36     }
37 }
```

Order.java

```
 1  package restaurant;
 2
 3  import java.util.List;
 4
 5  /** Submitted items and prices are snapshots; only lifecycle state changes. */
 6  public final class Order {
 7      public record Item(String description, int price) {
 8          public Item {
 9              if (description == null || description.isBlank() || price < 0)
10                  throw new IllegalArgumentException("Invalid order item");
11          }
12      }
13      private final int id;
14      private final List<Item> items;
15      private final String discountName;
16      private final int subtotal;
17      private final int discount;
18      private OrderState state = new OrderState.Pending();
19
20      Order(int id, List<Item> items, String discountName, int subtotal, int discount) {
21          this.id = id;
22          this.items = List.copyOf(items);
23          this.discountName = discountName;
24          this.subtotal = subtotal;
25          this.discount = discount;
26      }
27      void advance() { state = state.next(); }
28      public int id() { return id; }
29      public List<Item> items() { return items; }
30      public String discountName() { return discountName; }
31      public int subtotal() { return subtotal; }
32      public int discount() { return discount; }
33      public int total() { return subtotal - discount; }
34      public String status() { return state.label(); }
35      public boolean canAdvance() { return state.canAdvance(); }
36      public String toString() { return "#" + id + " | " + status() + " | " + total() + " KZT"; }
37  }
```

OrderBuilder.java

```
 1  package restaurant;
 2
 3  import java.util.ArrayList;
 4  import java.util.List;
 5  import java.util.Objects;
 6
 7  /** Builder: assemble item snapshots and a pricing policy before creating an Order. */
 8  public final class OrderBuilder {
 9      public record Totals(int subtotal, int discount) {
10          public int total() { return subtotal - discount; }
11      }
12
13      private final List<Order.Item> items = new ArrayList<>();
14      private DiscountStrategy strategy = new DiscountStrategy.None();
15
16      public OrderBuilder addMeal(Meal meal) {
17          Objects.requireNonNull(meal, "Meal is required");
18          items.add(new Order.Item(meal.description(), meal.price()));
19          return this;
20      }
21
22      public OrderBuilder withDiscount(DiscountStrategy strategy) {
23          this.strategy = Objects.requireNonNull(strategy, "Discount is required");
24          return this;
25      }
26
27      public Totals quote() {
28          int subtotal = 0;
29          for (Order.Item item : items) subtotal = Math.addExact(subtotal, item.price());
30          int discount = strategy.discount(subtotal);
31          if (discount < 0 || discount > subtotal) throw new IllegalArgumentException("Invalid discount");
32          return new Totals(subtotal, discount);
33      }
34
35      public Order build(int id) {
36          if (id < 1) throw new IllegalArgumentException("Order ID must be positive");
37          if (items.isEmpty()) throw new IllegalArgumentException("Add at least one item before
    submitting.");
38          Totals totals = quote();
39          return new Order(id, items, strategy.name(), totals.subtotal(), totals.discount());
40      }
41  }
```

OrderFacade.java

```
 1  package restaurant;
 2
 3  import java.util.List;
 4  import java.util.Objects;
 5
 6  /** Facade: coordinates the builder and order service behind simple UI operations. */
 7  public final class OrderFacade {
 8      public record Quote(int subtotal, int discount) {
 9          public int total() { return subtotal - discount; }
10      }
11      private final OrderService service;
12      public OrderFacade(OrderService service) { this.service = Objects.requireNonNull(service); }
13
14      public Quote quote(List<? extends Meal> meals, DiscountStrategy strategy) {
15          OrderBuilder.Totals totals = assemble(meals, strategy).quote();
16          return new Quote(totals.subtotal(), totals.discount());
17      }
18      private OrderBuilder assemble(List<? extends Meal> meals, DiscountStrategy strategy) {
19          Objects.requireNonNull(meals, "Order items are required");
20          OrderBuilder builder = new OrderBuilder().withDiscount(strategy);
21          for (Meal meal : meals) builder.addMeal(meal);
22          return builder;
23      }
24      public Order placeOrder(List<? extends Meal> meals, DiscountStrategy strategy) {
25          return service.submit(assemble(meals, strategy));
26      }
27  }
```

OrderObserver.java

```
1  package restaurant;
2
3  /** Observer: subscribers receive both submission and state-change events. */
4  @FunctionalInterface
5  public interface OrderObserver {
6      void orderChanged(Order order);
7  }
```

OrderService.java

```
 1  package restaurant;
 2
 3  import java.util.ArrayList;
 4  import java.util.List;
 5  import java.util.Objects;
 6
 7  /** In-memory order repository and Observer publisher. Used on Swing's event thread. */
 8  public final class OrderService {
 9      private final List<Order> orders = new ArrayList<>();
10      private final List<OrderObserver> observers = new ArrayList<>();
11      private int nextId = 1;
12
13      public void subscribe(OrderObserver observer) { observers.add(Objects.requireNonNull(observer)); }
14      public void unsubscribe(OrderObserver observer) { observers.remove(observer); }
15      public List<Order> orders() { return List.copyOf(orders); }
16      Order submit(OrderBuilder builder) {
17          Order order = builder.build(nextId);
18          nextId++;
19          orders.add(order);
20          publish(order);
21          return order;
22      }
23      public void advance(int id) {
24          Order order = orders.stream().filter(o -> o.id() == id).findFirst()
25              .orElseThrow(() -> new IllegalArgumentException("Order not found: " + id));
26          order.advance();
27          publish(order);
28      }
29      private void publish(Order order) {
30          for (OrderObserver observer : List.copyOf(observers)) observer.orderChanged(order);
31      }
32  }
```

OrderState.java

```
 1  package restaurant;
 2
 3  /** State: each concrete state defines its permitted successor. */
 4  public interface OrderState {
 5      String label();
 6      OrderState next();
 7      default boolean canAdvance() { return true; }
 8
 9      final class Pending implements OrderState {
10          public String label() { return "Pending"; }
11          public OrderState next() { return new Preparing(); }
12      }
13      final class Preparing implements OrderState {
14          public String label() { return "Preparing"; }
15          public OrderState next() { return new Ready(); }
16      }
17      final class Ready implements OrderState {
18          public String label() { return "Ready"; }
19          public OrderState next() { return new Completed(); }
20      }
21      final class Completed implements OrderState {
22          public String label() { return "Completed"; }
23          public boolean canAdvance() { return false; }
24          public OrderState next() { throw new IllegalStateException("This order is already completed.");
    }
25      }
26  }
```

RestaurantPanel.java

```
 1  package restaurant;
 2
 3  import javax.swing.*;
 4  import javax.swing.border.EmptyBorder;
 5  import java.awt.*;
 6  import java.util.ArrayList;
 7  import java.util.List;
 8
 9  /** Two views subscribe independently to the same order events. */
10  public final class RestaurantPanel extends JPanel {
11      private final OrderService service;
12      private final OrderFacade facade;
13      private final List<Meal> cart = new ArrayList<>();
14      private final JComboBox<MealCreator> menu = new JComboBox<>(new MealCreator[] {
15          new MealCreator.PizzaCreator(), new MealCreator.BurgerCreator(), new MealCreator.DrinkCreator()
16      });
17      private final JCheckBox cheese = new JCheckBox("Cheese +300 KZT");
18      private final JCheckBox sauce = new JCheckBox("Sauce +150 KZT");
19      private final JComboBox<DiscountStrategy> discounts = new JComboBox<>(new DiscountStrategy[] {
20          new DiscountStrategy.None(), new DiscountStrategy.Student(), new DiscountStrategy.Promotion()
21      });
22      private final DefaultListModel<String> cartModel = new DefaultListModel<>();
23      private final JList<String> cartList = new JList<>(cartModel);
24      private final JLabel preview = new JLabel();
25      private final JLabel totals = new JLabel();
26      private final JLabel notice = new JLabel("Build an order to get started.");
27      private final DefaultListModel<Order> customerModel = new DefaultListModel<>();
28      private final DefaultListModel<Order> kitchenModel = new DefaultListModel<>();
29      private final JList<Order> customerList = new JList<>(customerModel);
30      private final JList<Order> kitchenList = new JList<>(kitchenModel);
31      private final JTextArea customerDetails = detailsArea();
32      private final JTextArea kitchenDetails = detailsArea();
33      private final JButton advance = new JButton("Advance status");
34      private final JButton submit = new JButton("Submit order");
35      private final JTabbedPane tabs = new JTabbedPane();
36
37      public RestaurantPanel(OrderService service) {
38          super(new BorderLayout(0, 16));
39          this.service = service;
40          facade = new OrderFacade(service);
41          setBorder(new EmptyBorder(20, 24, 20, 24));
42          JLabel heading = new JLabel("Table");
43          JPanel header = new JPanel(new BorderLayout(20, 0));
44          header.add(heading, BorderLayout.WEST);
45          JPanel introduction = new JPanel(new GridLayout(2, 1, 0, 3));
46          introduction.add(new JLabel("Restaurant ordering"));
47          introduction.add(new JLabel("Choose your meal, add your extras, and follow your order."));
48          header.add(introduction, BorderLayout.CENTER);
49          header.setBorder(new EmptyBorder(0, 0, 6, 0));
50          add(header, BorderLayout.NORTH);
51          tabs.addTab("Place Order", customerTab());
52          tabs.addTab("Kitchen", kitchenTab());
53          add(tabs, BorderLayout.CENTER);
54          add(notice, BorderLayout.SOUTH);
55          UiTheme.apply(this);
56          heading.setFont(UiTheme.BODY.deriveFont(Font.BOLD, 36f));
57          notice.setForeground(UiTheme.MUTED);
58          notice.setBorder(new EmptyBorder(6, 0, 0, 0));
59          notice.setToolTipText("Orders are stored for this session only and clear when the app closes.");
60          preview.setFont(UiTheme.BODY.deriveFont(Font.BOLD));
61          totals.setFont(UiTheme.BODY.deriveFont(Font.BOLD));
62          UiTheme.primary(submit);
63          UiTheme.primary(advance);
64          service.subscribe(this::refreshCustomer);
65          service.subscribe(this::refreshKitchen);
66          menu.addActionListener(e -> {
67              boolean food = !(menu.getSelectedItem() instanceof MealCreator.DrinkCreator);
68              cheese.setEnabled(food);
69              sauce.setEnabled(food);
70              if (!food) { cheese.setSelected(false); sauce.setSelected(false); }
71              refreshPreview();
72          });
73          cheese.addActionListener(e -> refreshPreview());
74          sauce.addActionListener(e -> refreshPreview());
75          discounts.addActionListener(e -> refreshCart());
76          customerList.addListSelectionListener(e -> showDetails(customerList, customerDetails));
77          kitchenList.addListSelectionListener(e -> {
78              showDetails(kitchenList, kitchenDetails);
```

```
            Order selected = kitchenList.getSelectedValue();
            advance.setEnabled(selected != null && selected.canAdvance());
        });
        refreshPreview();
        refreshCart();
        for (Order order : service.orders()) { refreshCustomer(order); refreshKitchen(order); }
    }

    private JPanel customerTab() {
        JPanel left = new JPanel(new BorderLayout(0, 12));
        JPanel configuration = new JPanel(new GridLayout(0, 1, 0, 6));
        UiTheme.section(configuration, "1. Choose your meal");
        configuration.add(menu);
        JPanel extras = new JPanel(new GridLayout(1, 2));
        extras.add(cheese);
        extras.add(sauce);
        configuration.add(extras);
        configuration.add(preview);
        JButton add = new JButton("Add item to order");
        add.addActionListener(e -> { cart.add(configuredMeal()); refreshCart(); notice.setText("Item added. Add another or submit your order."); });
        configuration.add(add);
        left.add(configuration, BorderLayout.NORTH);
        JPanel basket = new JPanel(new BorderLayout(0, 8));
        UiTheme.section(basket, "2. Review your order");
        cartList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        basket.add(new JScrollPane(cartList), BorderLayout.CENTER);
        JPanel actions = new JPanel(new GridLayout(0, 1, 0, 8));
        JButton remove = new JButton("Remove selected item");
        remove.addActionListener(e -> {
            int index = cartList.getSelectedIndex();
            if (index >= 0) { cart.remove(index); refreshCart(); }
        });
        actions.add(remove);
        actions.add(discounts);
        actions.add(totals);
        actions.add(submit);
        submit.addActionListener(e -> submitCart());
        basket.add(actions, BorderLayout.SOUTH);
        left.add(basket, BorderLayout.CENTER);
        JPanel right = new JPanel(new BorderLayout(0, 12));
        UiTheme.section(right, "3. Track submitted orders");
        customerList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        JScrollPane history = new JScrollPane(customerList);
        history.setPreferredSize(new Dimension(0, 155));
        right.add(history, BorderLayout.NORTH);
        right.add(new JScrollPane(customerDetails), BorderLayout.CENTER);
        JPanel panel = new JPanel(new GridLayout(1, 2, 20, 0));
        panel.setBorder(new EmptyBorder(12, 0, 0, 0));
        panel.add(left);
        panel.add(right);
        return panel;
    }

    private JPanel kitchenTab() {
        JPanel panel = new JPanel(new BorderLayout(12, 12));
        panel.setBorder(new EmptyBorder(12, 0, 0, 0));
        panel.add(new JLabel("Pending  >  Preparing  >  Ready  >  Completed"), BorderLayout.NORTH);
        JPanel body = new JPanel(new GridLayout(1, 2, 20, 0));
        kitchenList.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        body.add(new JScrollPane(kitchenList));
        body.add(new JScrollPane(kitchenDetails));
        panel.add(body, BorderLayout.CENTER);
        advance.setEnabled(false);
        advance.addActionListener(e -> {
            Order order = kitchenList.getSelectedValue();
            if (order != null) {
                try { service.advance(order.id()); notice.setText("Order #" + order.id() + " is now " + order.status() + "."); }
                catch (IllegalArgumentException | IllegalStateException ex) { showError(ex.getMessage()); }
            }
        });
        panel.add(advance, BorderLayout.SOUTH);
        return panel;
    }

    private Meal configuredMeal() {
        Meal meal = ((MealCreator) menu.getSelectedItem()).prepare();
        if (cheese.isSelected()) meal = new Extra.Cheese(meal);
        if (sauce.isSelected()) meal = new Extra.Sauce(meal);
```

```
157         return meal;
158     }
159     private DiscountStrategy selectedDiscount() { return (DiscountStrategy) discounts.getSelectedItem(); }
160     private void refreshPreview() {
161         Meal meal = configuredMeal();
162         preview.setText("Item price: " + money(meal.price()));
163     }
164     private void refreshCart() {
165         cartModel.clear();
166         for (Meal meal : cart) cartModel.addElement(meal.description() + " | " + money(meal.price()));
167         OrderFacade.Quote quote = facade.quote(cart, selectedDiscount());
168         totals.setText("Subtotal " + money(quote.subtotal()) + " - " + money(quote.discount()) + " = " + money(quote.total()));
169         submit.setEnabled(!cart.isEmpty());
170     }
171     private void submitCart() {
172         try {
173             Order order = facade.placeOrder(cart, selectedDiscount());
174             cart.clear();
175             refreshCart();
176             customerList.setSelectedValue(order, true);
177             notice.setText("Order #" + order.id() + " submitted. Open Kitchen to begin preparation.");
178         } catch (IllegalArgumentException ex) { showError(ex.getMessage()); }
179     }
180     private void refreshCustomer(Order changed) {
181         refreshOrders(customerModel, customerList, changed);
182         showDetails(customerList, customerDetails);
183     }
184     private void refreshKitchen(Order changed) {
185         refreshOrders(kitchenModel, kitchenList, changed);
186         showDetails(kitchenList, kitchenDetails);
187         Order selected = kitchenList.getSelectedValue();
188         advance.setEnabled(selected != null && selected.canAdvance());
189     }
190     private void refreshOrders(DefaultListModel<Order> model, JList<Order> list, Order changed) {
191         Order selected = list.getSelectedValue();
192         model.clear();
193         for (Order order : service.orders()) model.addElement(order);
194         list.setSelectedValue(selected == null ? changed : selected, true);
195     }
196     private void showDetails(JList<Order> list, JTextArea area) {
197         Order order = list.getSelectedValue();
198         if (order == null) { area.setText("No order selected."); return; }
199         StringBuilder text = new StringBuilder("ORDER #" + order.id() + "\nStatus: " + order.status() + "\n\n");
200         for (Order.Item item : order.items()) text.append(item.description()).append("\n").append(money(item.price())).append("\n");
201         text.append("\nSubtotal: ").append(money(order.subtotal()))
202             .append("\nDiscount: ").append(order.discountName()).append("\nSaved: ").append(money(order.discount()))
203             .append("\nTOTAL: ").append(money(order.total()));
204         area.setText(text.toString());
205         area.setCaretPosition(0);
206     }
207     private void showError(String message) { JOptionPane.showMessageDialog(this, message, "Order issue", JOptionPane.WARNING_MESSAGE); }
208     private static String money(int value) { return String.format(java.util.Locale.US, "%,d KZT", value); }
209     private static JTextArea detailsArea() {
210         JTextArea area = new JTextArea("No order selected.");
211         area.setEditable(false);
212         area.setLineWrap(true);
213         area.setWrapStyleWord(true);
214         area.setMargin(new Insets(14, 14, 14, 14));
215         area.setFont(new Font(Font.SANS_SERIF, Font.PLAIN, 15));
216         return area;
217     }
218 }
```

UiTheme.java

```
 1  package restaurant;
 2
 3  import javax.swing.*;
 4  import javax.swing.border.EmptyBorder;
 5  import javax.swing.border.TitledBorder;
 6  import java.awt.*;
 7
 8  /** Small Swing-only visual theme; no business logic or external libraries. */
 9  final class UiTheme {
10      static final Color BACKGROUND = new Color(242, 246, 242);
11      static final Color INK = new Color(35, 65, 49);
12      static final Color MUTED = new Color(95, 111, 101);
13      static final Color LINE = new Color(213, 223, 215);
14      static final Color GREEN = new Color(38, 88, 62);
15      static final Color SELECTED = new Color(224, 238, 226);
16      static final Font BODY = new Font("Segoe UI", Font.PLAIN, 14);
17
18      private UiTheme() { }
19
20      static void section(JPanel panel, String title) {
21          panel.setBorder(BorderFactory.createCompoundBorder(
22              BorderFactory.createTitledBorder(BorderFactory.createLineBorder(LINE), title,
23                  TitledBorder.LEADING, TitledBorder.TOP, BODY.deriveFont(Font.BOLD), INK),
24              new EmptyBorder(10, 12, 12, 12)));
25      }
26
27      static void apply(Container root) {
28          UIManager.put("Button.gradient", null);
29          UIManager.put("Button.disabledText", MUTED);
30          style(root, BACKGROUND);
31      }
32
33      private static void style(Component component, Color background) {
34          component.setFont(BODY);
35          component.setForeground(INK);
36          if (component instanceof JPanel panel) {
37              // Sections are white; layout containers inherit their parent's surface.
38              boolean section = panel.getBorder() instanceof javax.swing.border.CompoundBorder;
39              background = section ? Color.WHITE : background;
40              panel.setBackground(background);
41          } else if (component instanceof JButton button) {
42              button.setBackground(SELECTED);
43              button.setFocusPainted(true);
44              button.setFont(BODY.deriveFont(Font.BOLD));
45              button.setBorder(BorderFactory.createCompoundBorder(
46                  BorderFactory.createLineBorder(LINE), new EmptyBorder(5, 12, 5, 12)));
47              button.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
48          } else if (component instanceof JCheckBox check) {
49              check.setBackground(background);
50          } else if (component instanceof JComboBox<?> combo) {
51              combo.setBackground(Color.WHITE);
52              combo.setBorder(BorderFactory.createLineBorder(LINE));
53              return; // Leave the look-and-feel's arrow-button layout intact.
54          } else if (component instanceof JList<?> list) {
55              list.setBackground(Color.WHITE);
56              list.setSelectionBackground(SELECTED);
57              list.setSelectionForeground(INK);
58              list.setFixedCellHeight(38);
59              list.setCellRenderer(new OrderRenderer());
60          } else if (component instanceof JTextArea area) {
61              area.setBackground(Color.WHITE);
62              area.setFont(BODY.deriveFont(15f));
63              area.setMargin(new Insets(16, 18, 16, 18));
64          } else if (component instanceof JScrollPane scroll) {
65              scroll.setBorder(BorderFactory.createLineBorder(LINE));
66              scroll.getViewport().setBackground(Color.WHITE);
67              Component view = scroll.getViewport().getView();
68              if (view != null) style(view, Color.WHITE);
69              return;
70          } else if (component instanceof JTabbedPane tabs) {
71              tabs.setBackground(BACKGROUND);
72              tabs.setFont(BODY.deriveFont(Font.BOLD));
73          }
74          if (component instanceof Container container) {
75              for (Component child : container.getComponents()) style(child, background);
76          }
77      }
78
```

```
    static void primary(JButton button) {
        primaryColors(button);
        button.addPropertyChangeListener("enabled", event -> primaryColors(button));
        button.setBorder(BorderFactory.createCompoundBorder(
            BorderFactory.createLineBorder(GREEN), new EmptyBorder(5, 14, 5, 14)));
    }

    private static void primaryColors(JButton button) {
        button.setBackground(button.isEnabled() ? GREEN : SELECTED);
        button.setForeground(button.isEnabled() ? Color.WHITE : MUTED);
    }

    /** Status remains written out; color is an additional cue, never the only cue. */
    private static final class OrderRenderer extends DefaultListCellRenderer {
        public Component getListCellRendererComponent(JList<?> list, Object value, int index,
                                                      boolean selected, boolean focused) {
            JLabel label = (JLabel) super.getListCellRendererComponent(list, value, index, selected,
    focused);
            label.setBorder(BorderFactory.createCompoundBorder(label.getBorder(), new EmptyBorder(6, 10,
     6, 8)));
            if (value instanceof Order order) {
                label.setFont(BODY.deriveFont(Font.BOLD));
                if (!selected) {
                    label.setForeground(switch (order.status()) {
                        case "Pending" -> new Color(126, 87, 24);
                        case "Preparing" -> new Color(50, 96, 136);
                        case "Ready" -> GREEN;
                        default -> MUTED;
                    });
                }
            }
            return label;
        }
    }
}
```